

DockFlow

Literature Review

Clothing Warehouse & Delivery Management System

Sec No-13 | Team No-14

U. Sindhuja • P. Bhagya Sri • S. Rishitha Reddy

Guided by Dr. M. Gayatri | Koneru Lakshmaiah Education Foundation



How this review covers every criterion



**Relevance to Project
Title**

10

MARKS

Slides 3-4

Four research areas linked to
DockFlow



**Depth & Quality of
Literature Review**

10

MARKS

Slides 5-7

8 sources, 2011-2026, each
summarized



**Critical Analysis &
Synthesis**

10

MARKS

Slides 8-9

Strengths, limits and common
themes



**Gap Analysis through
Literature**

10

MARKS

Slides 10-11

Gaps and how DockFlow
responds



Presentation

10

MARKS

Slides 1-13

Clear flow, references and
summary

REVIEW 2 TOTAL

5 criteria x 10 marks = 50

Four research areas behind our title

DockFlow: Clothing Warehouse & Delivery Management System

8 sources · 4 areas



Warehouse management systems

LINKED TO DOCKFLOW

Clothing warehouse: stock by size, color and location

Sources [2] [3]



SQL vs NoSQL databases

LINKED TO DOCKFLOW

MySQL for stock and orders;
MongoDB for delivery updates

Sources [6] [7]



Polyglot persistence

LINKED TO DOCKFLOW

Using more than one database inside one system

Sources [4] [5] [8]



Event streaming

LINKED TO DOCKFLOW

Kafka passing shipment events between services

Sources [1]

Every source linked to a DockFlow component

Source	Research area	Relevance to DockFlow
[1] Kreps et al., 2011	Event streaming	Basis for Kafka passing shipment events between services
[2] Redžeb, 2024	Warehouse systems	Current directions in warehouse management for retail
[3] Persson, thesis	Warehouse systems	Benefits of WMS: efficiency and real-time information
[4] Kazanavičius et al., 2022	Polyglot persistence	Using several data stores together in one system
[5] Halili et al., 2025	Polyglot persistence	Trade-offs of using several database types
[6] Future Internet, 2026	SQL vs NoSQL	When an SQL or a document database suits a workload
[7] EasyChair preprint, 2020	SQL vs NoSQL	MongoDB vs MySQL speed on basic operations
[8] Pllana & Krasniqi, 2019	Polyglot persistence	Each service using its own data store

Scope and quality of the review

8

sources reviewed

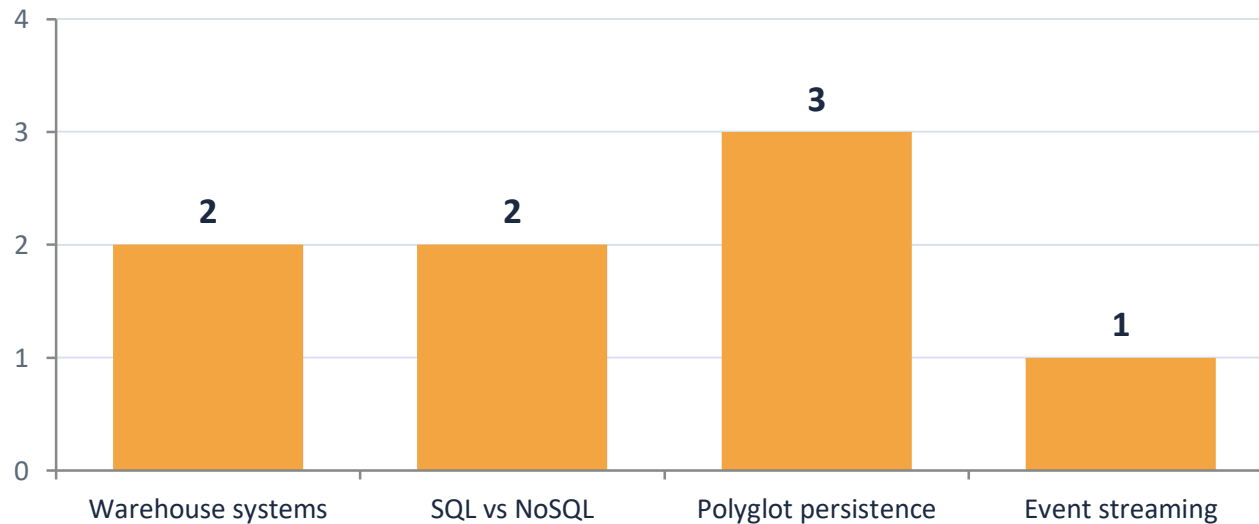
4

research areas

2011-2026

publication span

Sources reviewed per research area



SELECTION CRITERIA

- Directly linked to a DockFlow component
- Journal, conference or institutional source
- Recent where possible, plus one foundational paper
- Covers warehouse, database and messaging topics

Journal / conference: 5 | Preprint / thesis: 3

Warehouse systems and SQL vs NoSQL

[2] Redžeb (2024), Springer

STUDIED

Innovative warehouse management methods for omni-channel retail.

FOUND

Covers adapting warehouse operations, plus IoT and RFID in warehouse systems.

[3] Persson, Chalmers bachelor thesis

STUDIED

How warehouse management systems improve logistics, through a systematic literature review.

FOUND

WMS simplify processes and use real-time information; integration is a key challenge.

[6] Future Internet (2026)

STUDIED

SQL (indexed and not) vs a document database under read, write and mixed loads.

FOUND

Indexing matters for reads; the document store did better on write-heavy loads.

[7] EasyChair Preprint 4475 (2020)

STUDIED

MongoDB vs MySQL on insert, read, update and delete operations.

FOUND

MongoDB was faster on inserts, updates and deletes; reads were close. Authors favoured MongoDB.

Polyglot persistence and event streaming

[4] Kazanavičius et al. (2022), Applied Sciences

STUDIED

Moving a mainframe database to multi-model polyglot persistence in microservices.

FOUND

Proposed a migration approach; compared data quality using ISO/IEC 25012.

[5] Halili et al. (2025), arXiv

STUDIED

Polyglot persistence in microservices across relational, document, key-value, column-family and graph databases.

FOUND

More adaptability and domain fit, but more governance and operational complexity.

[8] Pllana & Krasniqi (2019), UBT Conference

STUDIED

Microservices where each service can use its own data store.

FOUND

Presents microservices as a route to polyglot persistence.

[1] Kreps, Narkhede & Rao (2011), NetDB

STUDIED

Kafka, a distributed messaging system for high-volume log data.

FOUND

Low-latency delivery; beat two popular messaging systems in tests; used in production at LinkedIn.

Critical analysis: strengths and limits

Source	Strength	Limitation for DockFlow
[1] Kreps et al., 2011	Tested in production at scale; clear design	Built for log data, not warehouse events; delivery is at-least-once, so duplicates must be handled
[2] Redžeb, 2024	Covers current WMS directions: IoT and RFID	Focus is on technology and hardware, with little on database design
[3] Persson, thesis	Systematic method; broad view of WMS benefits	A bachelor thesis that reviews literature; no system is built
[4] Kazanavičius et al., 2022	Uses a recognised data-quality standard	Case is a mainframe migration, not a warehouse
[5] Halili et al., 2025	Compares five database types on several criteria	General treatment; not specific to a warehouse and delivery system
[6] Future Internet, 2026	Controlled, containerized, reproducible tests	Single-node setup; shows trends, not absolute numbers
[7] EasyChair preprint, 2020	Simple, repeatable comparison of basic operations	Reports only operation speed; says nothing on consistency or transactions
[8] Pllana & Krasniqi, 2019	Explains why services can own separate data stores	Conceptual overview; no measured results in the abstract

Synthesis: what the sources say together



WMS add value

[2] [3]

Real-time information and efficiency gains, but integration is hard.



No single best database

[6] [7]

The better choice depends on the workload.



Polyglot has a price

[4] [5] [8]

Better fit per service, but more complexity to manage.



Events decouple services

[1]

Kafka moves data between services with low latency.

WHERE STUDIES DIFFER [7] favours MongoDB overall, while [6] finds the winner depends on indexing and workload. We follow [6].

FOR DOCKFLOW MySQL for stock and orders, MongoDB for flexible delivery updates, Kafka for shipment events, each with a narrow job.

Gaps in the literature and our response

Gap	What the literature shows	How DockFlow responds
Gap 1 Data design in WMS	WMS studies [2] [3] focus on IoT, RFID and implementation; database design is not their focus.	Database-first design: clothes tracked by size, color and location in MySQL.
Gap 2 Polyglot in warehouses	Polyglot studies [4] [5] [8] are general or migration-based, not warehouse and delivery.	MySQL for stock and orders, MongoDB for delivery updates, in one system.
Gap 3 Kafka for shipments	Kafka evidence [1] concerns log processing, not shipment events.	Kafka passes shipment events between the order and delivery services.
Gap 4 Evidence on trade-offs	Benchmarks [6] [7] are single-node or basic operations; consistency across stores is not examined.	Not yet addressed. DockFlow is a working course project; benchmarking is future work.

Gaps are based on the eight sources reviewed here; a wider review may reveal more.

Where DockFlow fits

Existing work covers warehouse systems, SQL vs NoSQL, polyglot persistence and Kafka separately. DockFlow brings them together for a clothing warehouse and its deliveries.



What we built

- Working course project
- MySQL, MongoDB and Kafka together
- Men's, women's and kids' clothes by size, color and location



What we do not claim

- No real courier GPS or payments
- No performance benchmark yet
- CI/CD and full C4 diagrams still to do



Suggested next steps

- Benchmark MySQL and MongoDB on DockFlow workloads
- Test duplicate shipment events with Kafka
- Widen the review: event streaming and 3PL

References

- [1] J. Kreps, N. Narkhede and J. Rao, "Kafka: a distributed messaging system for log processing," NetDB'11, Athens, Greece, 2011.
- [2] K. Redžeb, "Innovative Methods in Warehouse Management: A Comprehensive Review," in Technology: Toward Business Sustainability (ICBT 2023), LNNS vol. 925, Springer, Cham, 2024.
- [3] Persson, "The effective integration and optimization of Warehouse Management Systems: A Systematic Literature Review," Bachelor thesis, Chalmers University of Technology.
- [4] J. Kazanavičius et al., "An approach to migrate a monolith database into multi-model polyglot persistence based on microservice architecture: A case study for mainframe database," Applied Sciences, vol. 12, art. 6189, 2022.
- [5] F. Halili, A. Nuhiji and D. Mustafai-Veliu, "Polyglot Persistence in Microservices: Managing Data Diversity in Distributed Systems," arXiv:2509.08014, 2025.
- [6] "Benchmarking SQL and NoSQL Persistence in Microservices Under Variable Workloads," Future Internet, vol. 18, art. 53, 2026.
- [7] "MongoDB vs MySQL: A Comparative Study of MongoDB and MySQL Based on Their Performance," EasyChair Preprint 4475, Oct. 2020.
- [8] B. Pllana and G. Krasniqi, "Towards Polyglot Persistence: The Use of Various Data Storages with Microservices Architecture," UBT International Conference, 2019.

Summary: DockFlow Review 2



Relevant

Every source maps to a DockFlow component.



In depth

8 sources, 4 areas, 2011 to 2026, each summarized.



Critical

Strengths, limits and a contrast between studies.



Gap-driven

4 gaps, each linked to what DockFlow does.

Thank you

We are happy to take your questions.

U. Sindhuja • P. Bhagya Sri • S. Rishitha Reddy
Team No-14 | Guided by Dr. M. Gayatri